⚙ Conditions Nécessaires pour Réaliser un Bon Projet

Guide complet des conditions techniques, organisationnelles et matérielles pour que votre projet "Prix des carburants" soit un succès.

❶ CONDITIONS TECHNIQUES

🖥 Environnement de développement

PHP 8.x installé et fonctionnel

- PHP CLI accessible via terminal : `php -v`
- Extensions requises : curl, json, pdo (pour bases de données si besoin)
- Modèle d'erreurs configuré correctement
- Vérifier : `php -m` pour lister les extensions

Serveur web fonctionnel

- Apache 2 OU Nginx installé et running
- Support du PHP configuré correctement
- Répertoires publics accessibles (chmod 755)
- Tester : visiter `http://localhost` doit afficher une page
- VirtualHost configuré si besoin (domaine local)

Accès à Internet stable et rapide

- Tester les APIs régulièrement (essayer les requêtes cURL)
- Vérifier les rate limits des APIs (100 req/jour pour ipinfo.io free)
- Avoir un plan B si une API devient indisponible
- Tester la géolocalisation IP : `curl https://api.ipinfo.io/8.8.8.8`

Validateurs W3C accessibles

- Accès au validateur HTML : https://validator.w3.org/
- Accès au validateur CSS : https://jigsaw.w3.org/css-validator/
- Accès au validateur XML : https://www.w3.org/2001/03/webdata/xsv
- Outil d'analyse d'accessibilité : WAVE, axe DevTools

Éditeur de code performant

- VS Code OU PhpStorm OU Sublime Text

- Extensions pour PHP, HTML, CSS, JSON
- Colorisation syntaxe activée
- Git intégré (si pas en CLI)
- Live Server ou équivalent pour développement local

Outils de débogage

- DevTools navigateur (F12) pour debug client-side
- Xdebug pour PHP (optionnel mais utile)
- Logs PHP : vérifier /var/log/apache2/error.log ou équivalent
- var_dump() et print_r() pour déboguer PHP rapidement

🖥 Connaissances technologiques requises

Maîtrise HTML5

- Structure sémantique : header, nav, main, section, article, footer
- Formulaires : input, select, textarea, button
- Tags spécialisés : map, usemap, area (pour carte interactive)
- Attributs : id, class, data-*, aria-* pour accessibilité
- Validation HTML5 native (required, type, pattern)

Maîtrise CSS3

- Flexbox pour layouts modernes
- Grid pour mises en page complexes
- Media queries pour responsive design
- Animations et transitions
- Variables CSS (custom properties)
- Pseudo-éléments et pseudo-classes

Maîtrise PHP8

- Syntaxe de base : variables, conditions, boucles, fonctions
- Manipulations de tableaux et objets
- Requêtes HTTP : GET, POST, requête serveur→API
- Gestion des cookies et sessions
- Manipulation de fichiers (lecture/écriture CSV)
- Traitement JSON et XML : json_decode, simplexml
- POO (classes, héritage) pour code structuré

Compréhension des APIs REST

- Concept client-serveur-API (3-tiers)
- Requêtes HTTP (GET, POST, headers)
- Format JSON : structure, parsing, génération

- Format XML : structure, parsing, navigation
- Gestion des erreurs API (timeouts, 404, 500, rate limits)
- CORS et accès depuis le navigateur
- Clés API et authentification basique

Connaissances en JavaScript (bonus)

- Manipulation du DOM : getElementById, querySelector
- Event listeners : click, change, submit
- Fetch API ou XMLHttpRequest pour requêtes async
- LocalStorage et sessionStorage (ne pas confondre avec cookies)
- Permet d'améliorer l'UX avec validations côté client

Connaissance Doxygen

- Syntaxe des commentaires phpDoc : @param, @return, @throws
- Génération automatique de documentation HTML
- Commandes : `doxygen -g Doxyfile` puis `doxygen Doxyfile`
- Output en HTML dans le répertoire html/

🌐 APIs et ressources externes

Accès aux APIs requises

- **Ghibli API :** https://ghibliapi.vercel.app/ (gratuit, pas de clé)
- **Prix carburants :** https://www.prix-carburants.gouv.fr/rubrique/opendata/
- **API carburants complets :** https://data.economie.gouv.fr/api/explore/v2.1/...
- **Géolocalisation IP :** https://api.ipinfo.io/ (free : 100 req/jour)
- Tester CHAQUE API avant de commencer le projet
- Documenter les rate limits et les erreurs possibles

Données statiques (CSV)

- Télécharger les listes régions/départements/villes depuis data.gouv.fr
- Vérifier le format CSV : séparateurs, encodage UTF-8
- Stocker dans un répertoire du serveur (ex: /data/ ou /csv/)
- Charger les données au démarrage ou à la demande
- Ne pas mettre directement en dur dans le code PHP

Hébergement en ligne stable

- Plateforme recommandée : AlwaysData (cité dans le sujet)
- Ou équivalent : OVH, Hostinger, 1&1, SiteGround
- Spécifications minimum : PHP 8.x, 1GB disque, accès FTP/SSH
- Support des .htaccess et mod_rewrite (si besoin)
- Vérifier la limite de requêtes API externes

- Uptime minimum : 99% (voir SLA)

❷ CONDITIONS ORGANISATIONNELLES

👥 Organisation du binôme

Définition claire des rôles

- **Personne A :** Responsable API + Backend (PHP)
- **Personne B :** Responsable Frontend + UI/UX
- OU : répartition par parties (1, 2, 3, 4)
- Définir un "tech lead" ou responsable de l'intégration
- Documenter cette répartition dans le rapport final

Communication régulière

- Réunion hebdomadaire (même courte, 30 min)
- Canal de communication unique : Discord, Slack, ou même mail
- Partage de notes/décisions après chaque réunion
- Respecter les deadlines internes (avant les deadlines officielles)
- Anticiper les blocages : "Je suis bloqué en attente de X"

Versionning (Git obligatoire)

- Créer un dépôt Git : GitHub, GitLab, ou Gitea
- Structure : `main` (stable) + `dev` (en cours) + branches features
- Commits réguliers avec messages clairs : "Feat: ajout page tech API"
- Chaque personne travaille sur sa branche, merge à la fin
- Historique = preuve de travail partagé
- Backup automatique en cas de perte de données

Répartition équitable des tâches

- Éviter "1 personne fait tout"
- Chacun doit pouvoir présenter une part du projet
- Durée de parole équilibrée à la soutenance
- Documenter : "qui a fait quoi" dans le rapport

📅 Gestion du temps

Planning réaliste (et respecté)

- Semaine 15 (TD #11) : Partie 1 complète (API + film Ghibli + géoloc IP)
- Semaine 16-17 : Partie 2 (sélection région/dept/ville + affichage carburants)
- Semaine 18 : Partie 3 (stockage CSV + stats histogramme)

- Semaine 18 : Partie 4 (cookie jour/nuit)
- Semaine 18-19 : Tests, validation W3C, corrections bugs
- Avant 4 mai : Documentation, rapport, vidéo, packaging
- **Marges d'erreur :** Prévoir 20-30% de temps tampon

Validation progressive

- **TD #11 (semaine 15) :** Partie 1 validée par l'enseignant
- **Fin semaine 17 :** Vérifier que carburants + géoloc IP marchent
- **Fin semaine 18 :** Stocker CSV et afficher stats
- **Fin semaine 18 :** Cookie jour/nuit fonctionnel
- Ne pas attendre le dernier moment pour tester

Temps dédié minimum

- 4-5 heures par personne par semaine minimum
- 2-3 heures ensemble pour coordination
- Total : ~10-15 heures par semaine pour le binôme
- Éviter les "rushes" à la veille des deadlines

Buffer time (temps pour imprévus)

- API qui s'arrête ? 1 jour de secours
- Bug difficile à trouver ? 2 jours de secours
- Serveur en panne ? jour de secours
- Ne pas livrer le jour même de la deadline

❸ CONDITIONS DE QUALITÉ

✅ Code et architecture

Code structuré et lisible

- Indentation cohérente (4 espaces ou 1 tab, pas mélanger)
- Variables nommées clairement : $user_city, pas $x ou $tmp
- Fonctions courtes et une seule responsabilité
- Commentaires explicatifs (pas évident = exliquer)
- Pas de code commenté laissé traîner
- Pas de variable globale si possible

Architecture 3-tiers respectée

- **Couche présentation :** HTML/CSS/JS (navigateur)
- **Couche métier :** PHP (serveur) - récupère données, traite, formatte
- **Couche données :** APIs externes, fichiers CSV
- Client NE doit PAS appeler directement les APIs externes
- Toutes les requêtes doivent passer par votre serveur PHP

Gestion des erreurs

- Try/catch pour les requêtes API (timeout, 404, etc)
- Vérifier que JSON/XML est bien formé avant de parser
- Messages d'erreur clairs pour l'utilisateur
- Logs serveur pour déboguer en production
- Ne pas afficher les erreurs techniques à l'utilisateur final
- Prévoir des valeurs par défaut si API indisponible

Commentaires phpDoc complets

- Chaque fonction : description, @param, @return, @throws
- Chaque classe : description générale
- Exemple :
- ```
  /**
  * Récupère les stations à proximité d'une ville
  * @param string $city Nom de la ville
  * @param int $radius Rayon en km
  * @return array Liste des stations
  * @throws Exception Si API non disponible
  */
  ```

Séparation des fichiers

- Config centralisée : `config.php`
- Fonctions utiles : `lib/functions.php` ou `utils/`
  ```

- Requêtes API : `api/` ou `services/`
- Données : `data/` ou `csv/`
- Assets : `css/`, `js/`, `img/`
- Chaque page : `index.php`, `tech.php`, `stats.php`

🎨 Validation et standards

Validation W3C (HTML & CSS)

- **HTML5 :** Valider avec https://validator.w3.org/
- **CSS3 :** Valider avec https://jigsaw.w3.org/css-validator/
- 0 erreur, 0 warning avant de rendre
- Tester CHAQUE page du site
- Faire une capture d'écran des validations pour le rapport

Absence de bugs visibles

- Pas d'erreur PHP affichée sur la page (error_reporting désactivé en prod)
- Pas de console JavaScript avec erreurs (F12 → Console)
- Formulaires : validation côté serveur ET client
- Responsive design : tester sur mobile/tablet/desktop
- Navigateurs testés : Firefox, Chrome, Safari si Mac
- Tester avec les outils de débogage du navigateur

Accessibilité (WCAG 2.1)

- Texte alternatif sur les images : alt="description"
- Contraste suffisant (WCAG AA minimum)
- Outil : WAVE, axe DevTools, Lighthouse
- Testable au clavier (Tab, Enter, Echap)
- Heading hiérarchique : h1 → h2 → h3, pas de sauts
- Labels sur les champs de formulaire

Éco-conception (performance)

- Temps de chargement < 3 secondes (Lighthouse)
- Images optimisées : compressées, format webp si possible
- Minifier CSS et JS (optionnel mais recommandé)
- Éviter les requêtes API multiples inutiles
- Cache côté client si possible (headers HTTP)
- Tester avec Lighthouse dans les DevTools

Sécurité basique

- Pas de clé API en dur dans le code public
- Valider/nettoyer les entrées utilisateur : $_GET, $_POST

- Utiliser htmlspecialchars() pour afficher des données utilisateur
- Pas de requête SQL directe (si base de données)
- HTTPS sur l'hébergement final

📄 Documentation

Documentation du code (Doxygen)

- Générer avec : `doxygen -g Doxyfile`
- Configurer pour HTML output
- Exécuter : `doxygen Doxyfile`
- Résultat : dossier `html/index.html` navigable
- À inclure dans les livrables

Rapport projet (5-10 pages)

- **1. Couverture :** Titre, auteurs, date, université
- **2. Analyse :** Diagramme de cas d'utilisation
- **3. Conception :** Mockups, plan du site, architecture
- **4. Réalisation :** Choix techniques, APIs utilisées
- **5. Gestion :** Gantt, répartition des tâches
- **6. Tests :** Résultats validations, captures W3C
- **7. Conclusion :** Bilan et améliorations futures
- Format ODT/DOCX + PDF, numérotation automatique

README.md

- Noms des auteurs
- URL du site en ligne
- Instructions d'installation (comment faire tourner le code)
- Dépendances (PHP version, extensions, APIs)
- Structure des dossiers
- Credentials des APIs (sans clés privées !)

❹ CONDITIONS MATÉRIELLES

💻 Équipement

Ordinateur performant

- RAM : minimum 4GB (8GB recommandé)
- Processeur : multi-core moderne
- Stockage : SSD pour rapidité (5GB libre minimum)
- Permet d'exécuter serveur local + éditeur + navigateurs

Connexion Internet stable

- Débit : 5-10 Mbps minimum
- Fiable : pas de déconnexions fréquentes
- Nécessaire pour appels API et déploiement en ligne

Navigateurs multiples

- Firefox + Chrome installés
- Safari si Mac (tester cross-browser)
- Edge optionnel

🛠 Outils et logiciels

Outils requis (GRATUITS)

Git

https://git-scm.com/
Versionning de code

VS Code

https://code.visualstudio.com/
Éditeur de code

XAMPP/WAMP

Apache + PHP + MySQL
Serveur local

Doxygen

https://www.doxygen.nl/
Génération doc code

Postman

https://www.postman.com/
Test d'APIs

Figma

https://www.figma.com/
Mockups et design

Outils optionnels (recommandés)

- **FileZilla :** FTP/SFTP pour upload sur serveur
- **Inkscape :** Créer/éditer logos SVG
- **ImageOptim :** Compresser images
- **cURL :** Tester APIs en CLI
- **Lighthouse :** Inclus dans Chrome DevTools

🏢 Espace de travail

Environnement de travail

- Lieu calme, sans distractions prolongées
- Chaise confortable (risque RSI si assis 4-5h)
- Écran de taille correcte (au moins 15")
- Éclairage adéquat
- Lieu de travail partagé en binôme (physiquement si possible)

❺ CHECKLIST PRÉ-PROJET

✅ À vérifier AVANT de commencer

- ✓ PHP 8.x installé et fonctionnel : `php -v`
- ✓ Serveur web (Apache/Nginx) running et accessible
- ✓ Connexion Internet stable testée
- ✓ Git configuré avec un compte GitHub/GitLab
- ✓ VS Code (ou éditeur) avec extensions PHP/HTML/CSS
- ✓ Navigateurs (Firefox + Chrome) à jour
- ✓ Postman installé pour tester les APIs
- ✓ Créer le dépôt Git pour le projet
- ✓ Cloner le dépôt en local
- ✓ Créer la structure de dossiers de base

⚠ Vérifications des APIs

- ✓ Tester Ghibli API : `curl https://ghibliapi.vercel.app/films`
- ✓ Créer compte ipinfo.io et récupérer token (si utilisation premium)
- ✓ Tester prix-carburants API : vérifier le format (JSON/XML)
- ✓ Vérifier les rate limits et quotas gratuits
- ✓ Documenter les endpoints utilisés
- ✓ Vérifier les erreurs possibles (timeout, 404, etc)

✅ Réunion d'équipe initiale

- ✓ Décider qui fait quoi (rôles et responsabilités)

- ✓ Créer un planning détaillé (jalons hebdomadaires)
- ✓ Définir un lieu/heure de réunion régulière
- ✓ Choisir le nom du site et le logo
- ✓ Créer les mockups des pages principales (Figma)
- ✓ Définir la charte graphique (couleurs, fonts, style)
- ✓ Documenter toutes ces décisions

✕ Erreurs courantes à éviter

- **Ne pas valider à temps :** Laisser les bugs s'accumuler jusqu'au dernier moment
- **Répartition inégale :** Une personne fait tout, l'autre ne fait rien
- **Pas de versionning :** Perdre le code ou fusionner les modifications difficiles
- **API appelle directement du navigateur :** Violer l'architecture 3-tiers
- **Pas de gestion d'erreur API :** Site crash si API indisponible
- **Oublier la documentation :** Code = seul document du projet
- **Pas de tests W3C :** Livrer du HTML/CSS invalide
- **Ignorer les cookies :** Partie obligatoire (jour/nuit + ville)
- **Rapport bâclé :** Considérer qu'on peut écrire ça le jour même
- **Vidéo au dernier moment :** Besoin de temps pour scénariser et enregistrer

6 RESSOURCES ESSENTIELLES

📖 Documentation officielle

Ressource	URL	Utilité
PHP Manual	https://www.php.net/manual/en/	Référence complète PHP
MDN (HTML/CSS/JS)	https://developer.mozilla.org/	Référence web standards
W3C Validator	https://validator.w3.org/	Validation HTML
CSS Validator	https://jigsaw.w3.org/css-validator/	Validation CSS
Ghibli API Docs	https://ghibliapi.vercel.app/	Spécification API films
data.economie.gouv.fr	https://data.economie.gouv.fr/	API prix carburants
ipinfo.io Docs	https://ipinfo.io/	API géolocalisation IP

🎓 Tutoriels recommandés

- **JSON en PHP :** https://www.php.net/manual/en/function.json-decode.php
- **XML en PHP :** https://www.php.net/manual/en/class.simplexmlelement.php
- **Cookies en PHP :** https://www.php.net/manual/en/function.setcookie.php
- **cURL en PHP :** https://www.php.net/manual/en/book.curl.php
- **Formulaires HTML5 :** https://developer.mozilla.org/en-US/docs/Learn/Forms
- **CSS Flexbox :** https://css-tricks.com/snippets/css/a-guide-to-flexbox/
- **Git basics :** https://git-scm.com/book/en/v2
- **Doxygen tutorial :** https://www.doxygen.nl/manual/starting.html

---

📌 **Résumé : Un bon projet c'est 30% technique + 40% organisation + 30% qualité**

Document complet - Guide pour réaliser un projet web réussi

Généré pour le cours Développement Web L2-I S4 2025-2026